

UNIVERSITÉ DE KINHASA

Master 1 – Intelligence Artificielle

Implémentation générale de la méthode des nuées dynamiques

From scratch en Python

Étudiant :	KABONGO TSHIATA ETIENNE
Niveau :	M1 IA
Cours :	Science des données
Sujet :	Implémentation de l'algorithme des nuées dynamiques (from scratch)
Année académique :	2025–2026

Kinshasa, septembre 2026

Table des matières

1	Introduction	3
2	Problématique et objectifs	3
3	Fondements théoriques	3
3.1	Cadre général de Diday	3
3.2	Partition et centre de gravité	4
3.3	Distance et fonction objectif	4
3.4	Alternance des deux phases	4
4	Algorithme proposé	5
4.1	Pseudo-code	5
4.2	Critères d'arrêt et complexité	5
4.3	Choix de conception	5
5	Implémentation générale from scratch	6
5.1	Cas particulier du centroïde	6
6	Protocole expérimental	6
6.1	Jeu de données	6
6.2	Paramètres	6
7	Résultats et analyse	7
7.1	Partition obtenue	7
7.2	Convergence	8
7.3	Choix du nombre de classes	8
7.4	Validation comparative	9
7.5	Influence de la représentation	9
8	Application réelle au dataset Iris	10
8.1	Données et protocole	10
8.2	Résultats de segmentation	10
9	Limites et pistes d'amélioration	12
10	Reproductibilité et dépôt GitHub	12

11 Conclusion**12**

1 Introduction

L'apprentissage non supervisé cherche à extraire une structure à partir de données qui ne possèdent pas d'étiquette connue. Parmi ses tâches centrales, la classification automatique (*clustering*) consiste à constituer des groupes homogènes : les observations d'un même groupe doivent être proches, tandis que celles de groupes différents doivent être éloignées.

La méthode des nuées dynamiques a été proposée par Edwin Diday en 1971 [2]. Elle alterne une phase de représentation des classes et une phase d'affectation selon un critère d'adéquation. Sa portée est plus générale que celle des k -means : une classe peut être représentée par un point, plusieurs points, des axes factoriels, une distribution ou une structure adaptée au problème. Les travaux de MacQueen [7] et de Lloyd [6] correspondent au cas particulier des centroïdes.

Ce travail présente les fondements mathématiques, une implémentation générale en Python, une interface de sélection et une expérimentation reproductible. Les représentations, adéquations, affectations et critères d'arrêt sont programmés directement.

2 Problématique et objectifs

Soit une matrice $X \in \mathbb{R}^{n \times p}$. Comment partitionner ses observations sans imposer que chaque classe soit résumée par un unique centroïde ? L'enjeu est de construire un moteur commun qui adapte la représentation des nuées à la géométrie des données.

Les objectifs spécifiques sont :

- formaliser le problème d'optimisation et les étapes de l'algorithme ;
- coder toutes les opérations fondamentales sans KMeans externe ;
- gérer les classes vides et les initialisations défavorables ;
- observer la convergence et apprécier le choix de k ;
- vérifier séparément la cohérence des résultats avec une référence.

3 Fondements théoriques

3.1 Cadre général de Diday

Pour une partition $P = (C_1, \dots, C_k)$, la méthode fait intervenir une fonction de représentation g et une fonction d'adéquation d :

$$L_j = g(C_j), \quad z_i = \arg \min_{1 \leq j \leq k} d(\mathbf{x}_i, L_j).$$

Le représentant L_j n'est pas nécessairement un point. L'algorithme alterne la construction des représentants et l'affectation jusqu'à stabilisation du critère $J(P, L) = \sum_i d(\mathbf{x}_i, L_{z_i})$.

Table 1: Représentations intégrées dans l'application.

Mode	Représentant	Critère d'adéquation
Centroïde	moyenne μ_j	distance euclidienne au carré
Points	plusieurs médoïdes	distance au prototype le plus proche
Axes factoriels	centre et base principale	résidu orthogonal au sous-espace
Distribution	moyenne et covariance	log-vraisemblance gaussienne négative
Personnalisé	structure libre	fonction fournie par l'utilisateur

Lorsque L_j est réduit au centroïde, on retrouve les k -means. Ceux-ci sont donc un cas particulier des nuées dynamiques, et non leur définition complète.

3.2 Partition et centre de gravité

Une partition $P = (C_1, \dots, C_k)$ divise l'ensemble des observations en classes disjointes. Pour une classe non vide C_j , son centre de gravité est la moyenne de ses observations :

$$\mu_j = \frac{1}{|C_j|} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i. \quad (1)$$

Ce centre minimise la somme des distances euclidiennes au carré aux membres de la classe [5].

3.3 Distance et fonction objectif

La distance euclidienne entre une observation $\mathbf{x}_i$ et un centre μ_j est

$$d(\mathbf{x}_i, \mu_j) = \sqrt{\sum_{r=1}^p (x_{ir} - \mu_{jr})^2}. \quad (2)$$

Comme la racine carrée est croissante, l'affectation peut employer directement la distance au carré. L'inertie intra-classe à minimiser vaut

$$J(P, \mu) = \sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \mu_j\|_2^2. \quad (3)$$

3.4 Alternance des deux phases

À centres fixés, chaque observation rejoint la classe de son centre le plus proche :

$$z_i = \arg \min_{1 \leq j \leq k} \|\mathbf{x}_i - \mu_j\|_2^2. \quad (4)$$

À partition fixée, les centres sont recalculés par

$$\boldsymbol{\mu}_j^{(t+1)} = \frac{1}{|C_j^{(t)}|} \sum_{\mathbf{x}_i \in C_j^{(t)}} \mathbf{x}_i. \quad (5)$$

Chacune de ces phases ne peut augmenter l'objectif (3). Comme il existe un nombre fini de partitions, la procédure atteint un point fixe en un nombre fini d'étapes. Ce point est toutefois un optimum local, pas nécessairement l'optimum global.

4 Algorithme proposé

4.1 Pseudo-code

1. choisir le type de représentation des nuées ;
2. construire une partition initiale à partir de points dispersés ;
3. calculer $L_j = g(C_j)$ pour chacune des classes ;
4. calculer les adéquations $d(\mathbf{x}_i, L_j)$;
5. affecter chaque individu à la représentation la plus adéquate ;
6. réparer une éventuelle classe vide avec un individu mal représenté ;
7. calculer et mémoriser le critère global ;
8. arrêter si la partition ou le critère se stabilise ; sinon reprendre à l'étape 3 ;
9. répéter avec plusieurs initialisations et conserver le plus faible critère.

4.2 Critères d'arrêt et complexité

L'arrêt intervient lorsque deux partitions successives sont identiques ou que la variation absolue du critère devient inférieure à ε . Pour le mode centroïde et T itérations, le coût dominant est $O(Tnkp)$ et la matrice d'adéquation occupe $O(nk)$. Le coût augmente avec le nombre de prototypes, la décomposition factorielle ou l'inversion des covariances selon le mode choisi.

4.3 Choix de conception

L'implémentation utilise une graine pseudo-aléatoire pour assurer la reproductibilité. Plusieurs démarrages réduisent le risque d'un mauvais optimum local. En cas de classe vide, un individu mal représenté est réaffecté à cette classe. Une méthode `predict` applique ensuite les représentations apprises à de nouvelles observations.

5 Implémentation générale from scratch

La classe `NueesDynamiquesGenerales` sépare la construction d’une représentation (`_representer`) du calcul de son adéquation (`_adequation`). Elle offre les modes centroïde, points, axes, distribution et personnalisé. Cette architecture traduit directement le cadre de Diday, au lieu d’assimiler toute nuée à un centre de gravité.

L’interface `application.py` permet d’importer un CSV ou un fichier Excel, de choisir les variables numériques, k et la représentation. Elle restitue la partition, la convergence, une projection 2D et un CSV enrichi de la colonne `nuée`.

5.1 Cas particulier du centroïde

Le cœur du calcul des distances est le suivant :

```
1 def _distances_carre(X, centres):
2     differences = X[:, np.newaxis, :] - centres[np.newaxis, :,
3         :]
4     return np.sum(differences * differences, axis=2)
```

L’affectation et l’inertie ne nécessitent ensuite que l’indice du minimum de chaque ligne et la somme des distances retenues :

```
1 distances = self._distances_carre(X, centres)
2 etiquettes = np.argmin(distances, axis=1)
3 inertie = np.sum(distances[np.arange(X.shape[0]), etiquettes])
```

NumPy sert ici uniquement à représenter les tableaux et à effectuer les opérations arithmétiques. La logique de clustering est entièrement contenue dans la classe `NueesDynamiques`. Le fichier complet est fourni dans `src/nuees_dynamiques.py`.

6 Protocole expérimental

6.1 Jeu de données

Le jeu synthétique contient 450 observations bidimensionnelles, réparties en trois nuages gaussiens de 150 points. Leurs moyennes génératrices sont $(-3, -2)$, $(0, 2, 3, 2)$ et $(3, 6, -0, 8)$, avec des dispersions différentes. Une graine fixée à 42 permet de régénérer exactement les mêmes données. Le CSV produit est inclus dans le projet.

6.2 Paramètres

L’expérience principale emploie $k = 3$, au maximum 200 itérations, une tolérance de 10^{-6} et 20 initialisations. L’étude du coude compare ensuite les valeurs $k = 1, \dots, 8$ avec 15 initialisations pour chacune.

7 Résultats et analyse

7.1 Partition obtenue

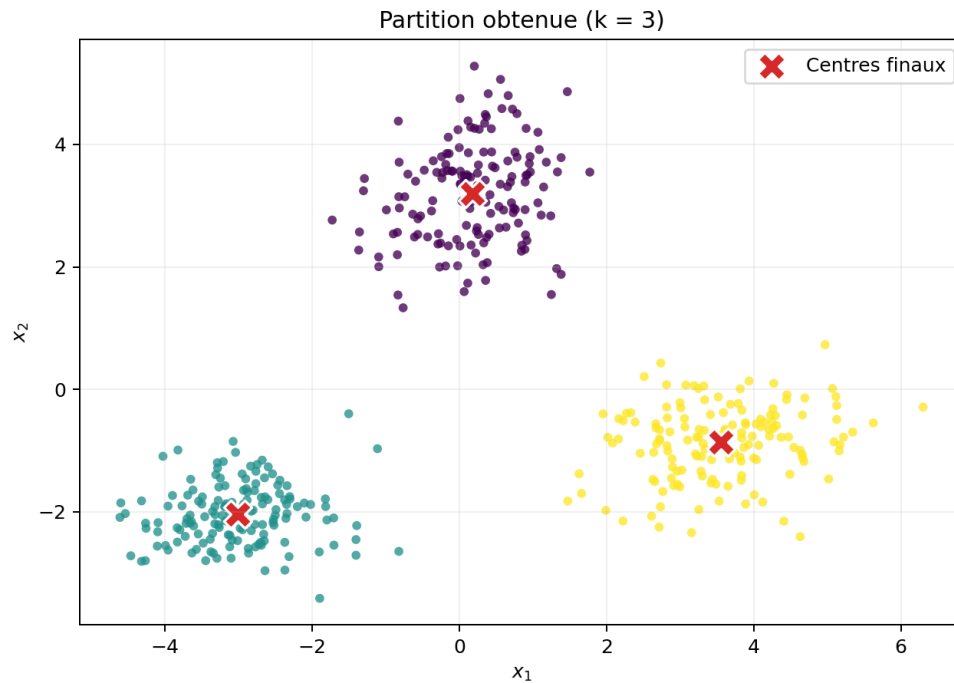


Figure 1: Classes découvertes et centres de gravité finaux.

Les trois groupes de la figure 1 sont nettement séparés. Les centres calculés sont proches des moyennes génératrices :

$$(0,183; 3,203), \quad (-3,014; -2,035), \quad (3,554; -0,862).$$

L'ordre des classes n'a pas de signification : une permutation des indices décrit la même partition.

7.2 Convergence

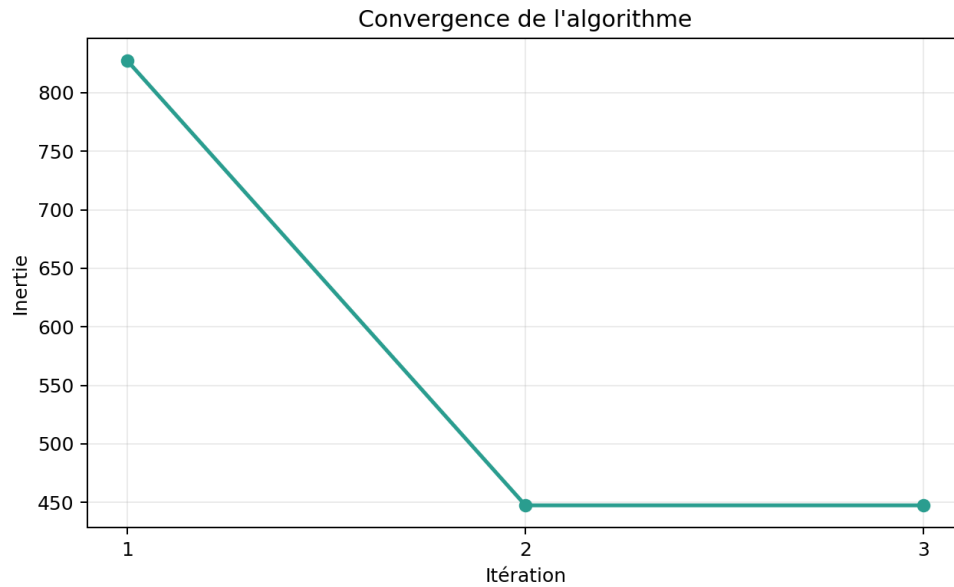


Figure 2: Évolution de l'inertie de la meilleure exécution.

La meilleure exécution atteint une inertie de 447,520831 en deux itérations. La décroissance puis la stabilisation observées sont conformes au principe d'optimisation alternée.

7.3 Choix du nombre de classes

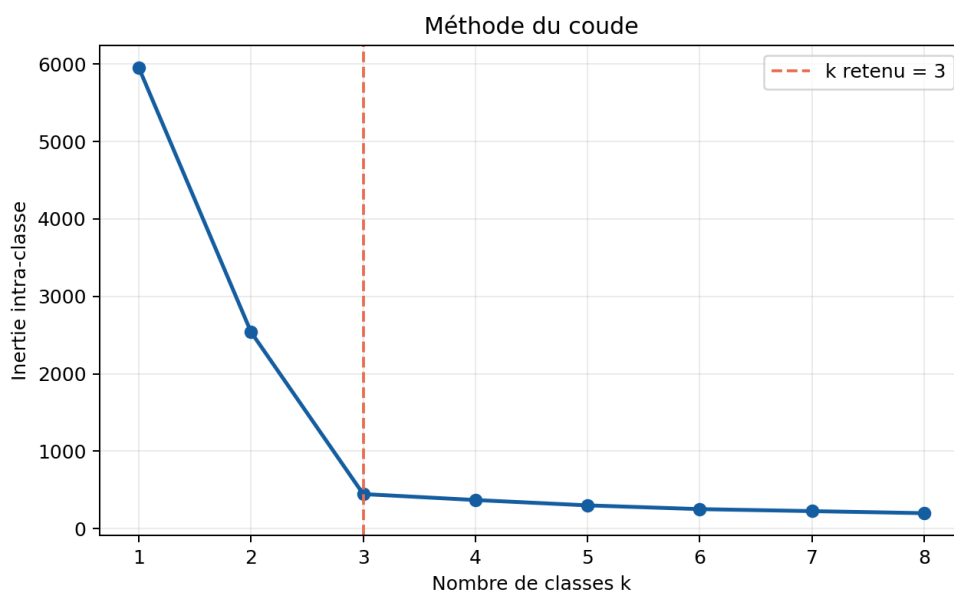


Figure 3: Inertie en fonction du nombre de classes.

Le passage de $k = 2$ (2538,877) à $k = 3$ (447,521) apporte un gain majeur. Après $k = 3$, les diminutions deviennent beaucoup plus progressives. Le coude confirme donc le choix de trois classes pour ce jeu de données.

7.4 Validation comparative

La comparaison n'intervient qu'après l'exécution du code personnel. Avec les mêmes données, l'implémentation de référence décrite dans la documentation de scikit-learn [8] obtient elle aussi une inertie de 447,520831. L'indice de Rand ajusté (ARI) entre les deux partitions vaut 1,000, de même que l'ARI entre notre partition et les classes génératrices. Cette égalité montre, pour l'expérience considérée, que les partitions sont identiques à une permutation des étiquettes près.

7.5 Influence de la représentation

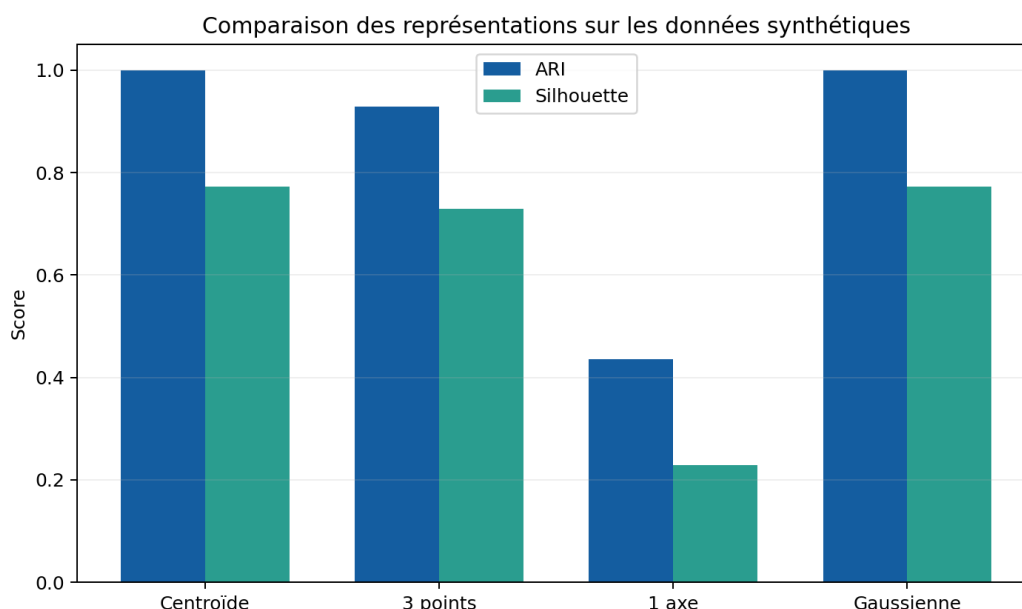


Figure 4: ARI et silhouette selon la représentation choisie.

Sur ces groupes gaussiens compacts, le centroïde et la distribution retrouvent parfaitement les classes ($\text{ARI} = 1$). Trois points représentatifs donnent un ARI proche de 0,929. Le mode axial est moins pertinent (ARI proche de 0,435), car les classes simulées sont compactes plutôt qu'organisées le long de droites. La flexibilité de la méthode permet précisément de comparer ces hypothèses et de retenir celle qui correspond aux données.

Table 2: Synthèse quantitative de l'expérience.

Indicateur	Valeur
Nombre d'observations	450
Nombre de variables	2
Nombre de classes	3
Itérations de la meilleure exécution	2
Inertie finale	447,520831
ARI avec scikit-learn	1,000000
ARI avec les classes génératrices	1,000000

8 Application réelle au dataset Iris

8.1 Données et protocole

Le dataset Iris, rendu célèbre par Fisher [3] et diffusé par le référentiel UCI [4], contient 150 fleurs : 50 *setosa*, 50 *versicolor* et 50 *virginica*. Chaque fleur est décrite par quatre mesures en centimètres : longueur et largeur du sépale, longueur et largeur du pétale.

Les quatre variables sont standardisées avant segmentation. La colonne `species` est strictement exclue de l'apprentissage ; elle ne sert qu'après coup au calcul de l'ARI. Le nombre de nuées est fixé à trois pour vérifier dans quelle mesure la structure non supervisée retrouve les trois espèces. Chaque représentation est exécutée avec 20 initialisations.

8.2 Résultats de segmentation

Table 3: Résultats des représentations sur Iris avec $k = 3$.

Représentation	ARI	Silhouette	Itérations
Centroïde (k-means)	0,620	0,460	5
Trois points	0,389	0,309	3
Un axe factoriel	0,535	0,333	15
Distribution gaussienne	0,941	0,381	14

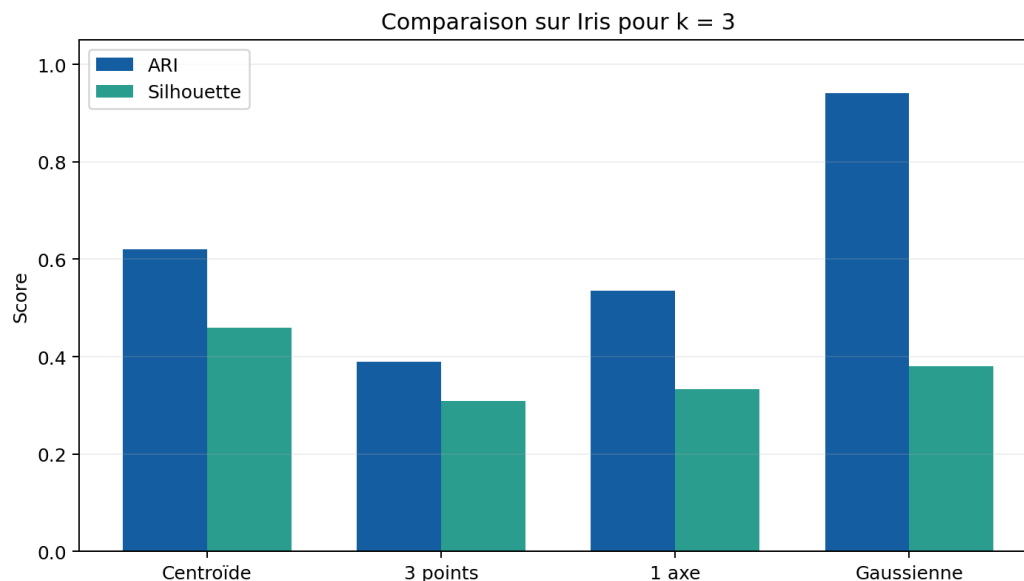


Figure 5: Comparaison des représentations sur Iris.

La distribution gaussienne obtient le meilleur accord avec les espèces réelles ($ARI = 0,941$). Le centroïde atteint 0,620. Ce gain confirme l'intérêt des nuées dynamiques : une classe caractérisée par une moyenne et une covariance peut décrire sa forme et sa dispersion, alors qu'un centroïde ne conserve qu'un point central.

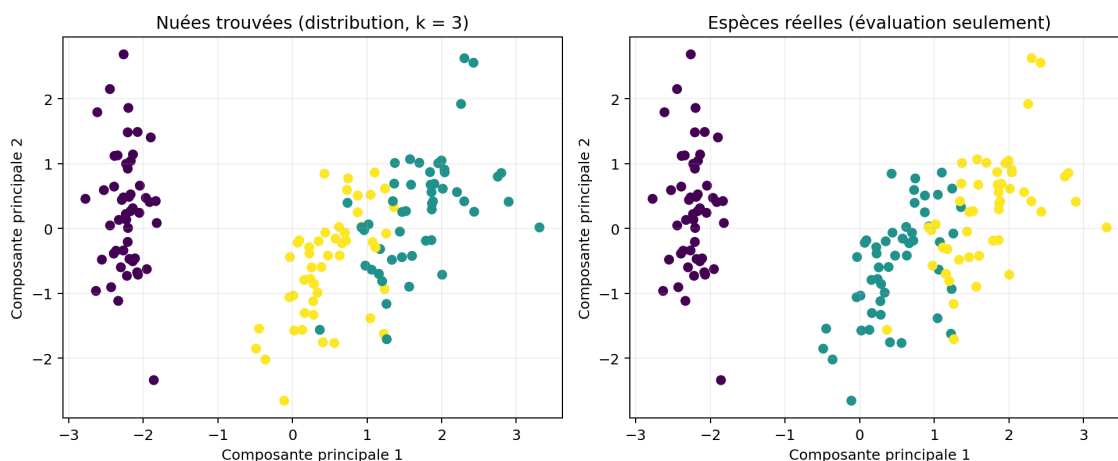


Figure 6: Projection ACP : segmentation gaussienne et espèces réelles.

La projection montre que *setosa* forme un groupe très distinct. Les deux autres espèces se chevauchent davantage. Pour cette raison, l'indice de silhouette atteint son maximum global avec deux macrogroupes, tandis que la contrainte $k = 3$ et l'ARI révèlent que la distribution gaussienne restitue très bien les trois espèces. Les classes réelles servent ici à l'évaluation, jamais à la construction des nuées.

9 Limites et pistes d'amélioration

L'algorithme exige que k soit fixé à l'avance et reste sensible à l'initialisation, malgré les redémarrages. La distance euclidienne le rend plus adapté aux classes compactes et approximativement sphériques ; il décrit mal les formes non convexes. Les variables de grande échelle peuvent dominer les distances, d'où la nécessité de standardiser les données réelles. Enfin, les valeurs aberrantes déplacent les moyennes.

Des extensions possibles comprennent l'initialisation k -means++ [1], la standardisation automatique, l'indice de silhouette pour guider le choix de k , le traitement par mini-lots pour de grands volumes et la comparaison avec DBSCAN ou un clustering hiérarchique.

10 Reproductibilité et dépôt GitHub

Le projet contient le code, les tests, les données, les figures et les sources LaTeX. Après téléchargement, l'expérience s'exécute par :

```
pip install -r requirements.txt
python experiments/experimentation.py
pytest -q
streamlit run application.py
```

Le dépôt public du projet est :

<https://github.com/THEETIENNE/nuees-dynamiques-from-scratch>

11 Conclusion

Ce travail implémente les nuées dynamiques dans leur sens général. Le moteur alterne représentation et affectation et accepte un centroïde, plusieurs prototypes, un sous-espace factoriel, une distribution ou une structure personnalisée. L'interface rend ce choix accessible à l'utilisateur. L'étude montre que la représentation doit être choisie selon la géométrie des données. Sur Iris, la représentation gaussienne atteint un ARI de 0,941 contre 0,620 pour le centroïde. Ce résultat fournit une démonstration empirique de la généralité et de l'utilité du cadre proposé par Diday.

Références

- [1] David Arthur and Sergei Vassilvitskii. k -means++: The advantages of careful seeding. *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.
- [2] Edwin Diday. Une nouvelle méthode en classification automatique et reconnaissance des formes : la méthode des nuées dynamiques. *Revue de Statistique Appliquée*, 19(2):19–33, 1971.

- [3] R. A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.
- [4] R. A. Fisher. Iris dataset, 1988.
- [5] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2 edition, 2009.
- [6] Stuart P. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.
- [7] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297. University of California Press, 1967.
- [8] Scikit-learn developers. *KMeans Documentation*, 2026. Consulté le 19 septembre 2026.